COS 201
COMPUTER PROGRAMMING I
LECTURE NOTE: WEEKS 1–3

Week 1: Introduction to Programming and Problem-Solving
Week 2: Development Environment and First Java Program
Week 3: Variables, Data Types, Expressions and Input/Output

Lecturer: Dr. Ajilore Opeoluwa
Target programmes: CSC, CYB, INS, SEN and ICH
Programming language: Java

Contents and note structure

- Week 1 — Introduction to Programming and Problem-Solving
- Week 2 — Development Environment and First Java Program
- Week 3 — Variables, Data Types, Expressions and Input/Output
- Exercises, practical code and GitHub submission procedures

Week 1: Introduction to Programming and Problem-Solving

Target students: CSC, CYB, INS, SEN and ICH
Programming language: Java
Main focus: Understanding programming, analysing problems, designing algorithms, using pseudocode and flowcharts, and preparing for practical programming.

1. Learning objectives

By the end of this lecture, students should be able to:
Define a computer programme, programming and problem-solving.
Explain why programming is important in different computing disciplines.
Identify the major stages of program development.
Explain the difference between an algorithm, pseudocode, a flowchart and source code.
Identify the input, processing and output of a simple computing problem.
Design an algorithm to solve a basic arithmetic or information-processing problem.
Write pseudocode and draw a flowchart for a simple problem.
Identify common characteristics of a good algorithm.
Write and run a simple Java programme.
Create a GitHub repository, commit source code and submit a repository link to the lecturer.

2. Introduction to computer programming

2.1 What is a computer?



A computer is an electronic device that accepts data, processes it according to instructions, stores data or results, and produces useful output.
For example, when a lecturer calculates students' examination results using a computer:
Input: Students' scores.
Processing: Addition, averaging and grading.
Output: Total scores, averages and grades.
Storage: Saving the results for future use.
A computer does not automatically know how to solve every problem. It requires instructions, software and appropriate data.

2.2 What is a computer programme?



A computer programme is a set of instructions written in a programming language to make a computer perform a particular task.

Examples include:

A programme that calculates students' examination results.

An application that manages university records.

A programme that calculates a customer's bill.

A tool that analyses network logs for suspicious activities.

A system that manages products in a shop.

An application that helps users organise information.

A programme can contain instructions for accepting data, performing calculations, making decisions, repeating operations and displaying results.

2.3 What is computer programming?



Computer programming is the process of designing, writing, testing, debugging and maintaining instructions that a computer can execute.

Programming involves more than typing code. A programmer must first understand the problem, determine the expected result, develop a logical solution, translate the solution into a programming language and test whether the programme works correctly.

Consider this problem:

A lecturer wants to calculate the average of three students' scores: 60, 70 and 80.

The solution requires the following steps:

Accept or identify the three scores.

Add the scores together.

Divide the total by three.

Display the total and average.

The total is 60+70+80=210.

The average is 210/3=70.

A programme can automate this process so that the lecturer does not have to repeat the calculation manually.

3. Why is programming important?

Programming is a foundation for developing software, automating tasks, processing information and solving computational problems.

Computer Science (CSC)

Programming helps students understand algorithms, data structures, artificial intelligence, databases and computational problem-solving.
Example: Developing a programme that searches for a particular number in a list.

Cybersecurity (CYB)

Programming supports security automation, log analysis, input validation and the development of defensive security tools.
Example: Writing a programme to count repeated failed login attempts in a sample log file.

Information Systems (INS)

Programming helps automate business processes and transform raw data into useful information.
Example: Creating a programme that calculates sales totals and produces a simple report.

Software Engineering (SEN)

Programming is essential for implementing, testing, debugging and maintaining software systems.
Example: Developing and testing a student-record application.

Industrial Chemistry (ICH)

Programming can support digital information processing, automation and the organisation of information relevant to the programme's specific field.
Example: Creating a programme that organises records or processes information systematically.
The examples illustrate possible applications; the particular ICH applications should be adapted to the official meaning and curriculum of the programme at your institution.

4. Programming languages and programming paradigms

4.1 What is a programming language?
A programming language is a formal language used to express instructions that a computer system can process.
Examples include:
For this course, we will use Java to introduce programming fundamentals and object-oriented concepts.

4.2 What is a programming paradigm?
A programming paradigm is a general approach or style used to organise and express a programme.

1. Procedural programming

Organises instructions into procedures or functions that perform specific tasks.
Example languages: C and procedural-style Java.
Example: A sequence of instructions for calculating a student's average.

2. Object-oriented programming (OOP)

Organises a programme around classes and objects that combine data and behaviour.
Example languages: Java and C++.
Example: A Student object with a name, score and method for calculating a grade.

3. Functional programming

Emphasises functions, expression evaluation and transformations of data.
Example languages: Haskell and functional-style programming in Java or Python.
Example: Applying a function to calculate the square of each number in a list.

4. Logic programming

Represents knowledge using facts and rules, then uses logical inference to derive results.
Example language: Prolog.

Example: Representing relationships and querying which individuals satisfy specified rules.
Important: A programming language may support more than one paradigm. Java, for example, is strongly associated with object-oriented programming but also supports functional-style features.

5. How does a computer execute a programme?

A computer needs instructions represented in a form its execution environment can process.
In Java, the basic process is:
Java source code
HelloWorld.java
Java compiler (javac)
Checks and compiles the source code
Bytecode
HelloWorld.class
Java Virtual Machine (JVM)
Executes the bytecode
Program output
Displayed on the screen
The Java Development Kit (JDK) provides the compiler and development tools. The JVM is responsible for executing Java bytecode.

6. The program development life cycle

A programme should be developed systematically rather than by writing random instructions and hoping they work.
The major stages are explained below.

1. **Problem definition**

Identify exactly what needs to be solved. Example: calculate the average of three examination scores.

2. **Problem analysis**

Identify the required inputs, processing steps, expected outputs and constraints.

3. **Solution design**

Develop an algorithm and represent it using pseudocode or a flowchart.

4. **Coding**

Translate the designed solution into a programming language such as Java.

5. **Compilation and execution**

Compile the source code and run the programme.

6. **Testing and debugging**

Try appropriate test cases, identify errors and correct them.

7. **Documentation**

Explain the purpose of the programme, how it works and how to run it.

8. **Maintenance**

Correct defects, adapt to new requirements and improve the programme over time.
Example: Applying the development stages
Suppose a department needs a programme to calculate the average score of a student.
These stages are related, but software development is often iterative. Testing may reveal a design problem, requiring the programmer to revisit an earlier stage.

7. Understanding the Input–Process–Output (IPO) model

The IPO model is a simple method for analysing a problem before writing a programme.
Input: The data supplied to the programme.
Process: The calculations or operations performed on the data.
Output: The information produced by the programme.

Example 1: Calculating a student's average
Problem: Calculate the average of three scores: 60, 70 and 80.
INPUT
Scores: 60, 70, 80

PROCESS
Total=60+70+80=210
Average=210/3=70
OUTPUT
Total = 210
Average = 70

Example 2: Calculating the total cost of items
A student buys four exercise books at ₦500 each.
Input: Quantity = 4; unit price = ₦500.
Process: Total cost = quantity × unit price.
Output: ₦2,000.
The mathematical expression is:
Total Cost = Quantity × Unit Price

EXERCISE 1
Identify the input, process and output for each problem.

1. Calculate the area of a rectangle.
2. Calculate a customer's total shopping bill.
3. Determine whether a number is even or odd.
4. Calculate the total failed login attempts in a sample security log.
5. Calculate the average monthly sales of a business.

8. What is an algorithm?

An algorithm is a finite, ordered sequence of clear instructions designed to solve a problem or complete a task. Algorithms are not limited to computers. A cooking recipe, instructions for withdrawing money from an ATM, or a method for calculating an average can be described as an algorithm.

Example: Algorithm for adding two numbers

Start.
Read the first number, A.
Read the second number, B.
Calculate SUM = A + B.
Display SUM.
Stop.

If A = 12 and B = 8, the output is 20.

Characteristics of a good algorithm
Finiteness: The algorithm must terminate after a finite number of steps for inputs within its intended scope.
Definiteness: Each instruction must be clear and unambiguous.
Input: The algorithm may accept zero or more clearly defined inputs.
Output: It must produce the required result for valid inputs.
Effectiveness: Its operations must be sufficiently simple and executable.
Correctness: The algorithm must produce the intended result for all inputs within its stated requirements.

Example of a poorly designed algorithm
Consider the following instructions:

Start.
Read two numbers.
Calculate the result.
Display the answer.
Stop.

The problem is that “calculate the result” does not specify which operation to perform. The instructions are ambiguous.

A better algorithm specifies whether to add, subtract, multiply or divide the numbers.

9. What is pseudocode?

Pseudocode is an informal, language-independent way of expressing an algorithm using structured, readable statements that resemble programming instructions.
Pseudocode is useful because students can concentrate on the logic of a solution before learning the exact syntax of a programming language. It does not have one universally compulsory syntax.

Example 1: Adding two numbers

```
START
INPUT A
INPUT B
SUM = A + B
OUTPUT SUM
STOP
```

Example 2: Calculating an average

```
START
INPUT score1
INPUT score2
INPUT score3
total = score1 + score2 + score3
average = total / 3
OUTPUT total
OUTPUT average
STOP
```

Example 3: Determining whether a student passed

Assume that the pass mark is 50 and valid scores are between 0 and 100.

```
START
INPUT score
IF score < 0 OR score > 100 THEN
OUTPUT "Invalid score"
ELSE IF score >= 50 THEN
OUTPUT "Pass"
ELSE
OUTPUT "Fail"
END IF
STOP
```

This algorithm first validates the score. It then decides whether the student has passed.

Example 4: Finding the larger of two numbers

```
START
INPUT A
INPUT B
IF A > B THEN
OUTPUT A
ELSE IF B > A THEN
OUTPUT B
ELSE
OUTPUT "Both numbers are equal"
END IF
STOP
```

Class discussion: Why is it necessary to include an equality case? Because if A and B have the same value, neither is larger than the other.

10. What is a flowchart?

A flowchart is a graphical representation of an algorithm using standard symbols connected by arrows to show the order of execution. Flowcharts help programmers visualise a solution before writing source code.

Common flowchart symbols

Terminator: An oval or rounded shape representing the start or end.

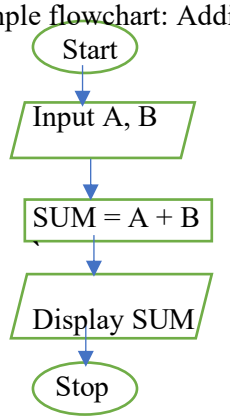
Process: A rectangle representing an operation or calculation.

Decision: A diamond representing a condition with possible branches, such as Yes/No.

Input/output: A parallelogram representing data entry or displaying results.

Flow line: An arrow showing the direction of execution.

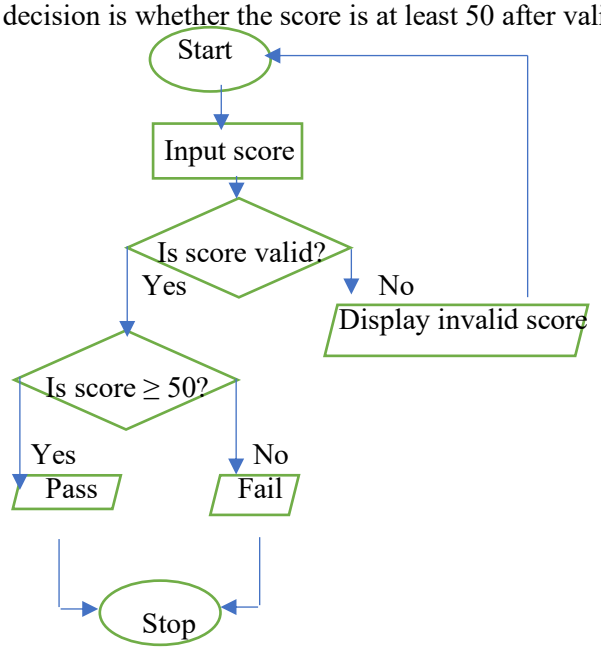
Example flowchart: Adding two numbers



When drawing a formal flowchart, use the correct graphical symbols: terminators for Start and Stop, parallelograms for input/output, rectangles for processes and arrows for flow direction.

Example flowchart: Pass or fail

The decision is whether the score is at least 50 after validation.



In a formal flowchart, each branch should be connected using arrows, and the decision branches should be labelled clearly. All branches should eventually reach the appropriate termination point.

11. Comparing algorithms, pseudocode, flowcharts and source code

These four representations complement one another. A programmer may begin with a written algorithm, express it as pseudocode, draw a flowchart and then write source code.

12. Common programming errors

Errors are normal in programming. Students should learn to identify and correct them systematically.

12.1 Syntax errors

These occur when source code violates the language's rules.
Example:

```
System.out.println("Hello")
```

The statement is missing a semicolon. The corrected version is:

```
System.out.println("Hello");
```

12.2 Runtime errors

These occur while a programme is running.
Example:

```
int a = 10;
int b = 0;
System.out.println(a / b);
```

Java throws an ArithmeticException for integer division by zero.

12.3 Logical errors

A programme may compile and run but produce the wrong result.
Incorrect example:

```
int length = 10;
int width = 5;
int area = length + width;
System.out.println(area);
```

The programme adds the length and width instead of multiplying them.
Correct version:

```
int length = 10;
int width = 5;
int area = length * width;
System.out.println(area);
```

The correct area is 50 square units.

12.4 How to debug a programme

Read the error message carefully.
Identify the line where the error occurs.
Check variable names, operators, brackets and semicolons.
Review the algorithm and compare it with the intended solution.
Test with small, known inputs.
Correct the error and run the programme again.
Confirm that the corrected programme produces the expected output.
Do not make random changes to code without understanding the problem. Debugging should be a logical investigation.

13. Week 1 practical: Write your first Java programme

This practical introduces students to source code, the Java compiler, program execution and basic output.

Practical objective
By the end of the exercise, students should be able to create a Java source file, compile it, run it, interpret the output and prepare it for submission through GitHub.

Step 1 — Prepare your working folder
Create a folder on your computer named:
COS201-Programming-I
Inside it, create another folder named:
week01
The folder structure should look like this:
COS201-Programming-I/
└── week01/
If you have not yet installed the JDK, Visual Studio Code and Git, complete the installation instructions provided for Week 2 before attempting the computer-based practical. The algorithm-design exercises in this lecture can be completed without installing any software.

Step 2 — Create your first Java source file
Open Visual Studio Code.
Select File → Open Folder.
Open your COS201-Programming-I folder.
In the Explorer panel, open week01.
Create a new file named FirstProgram.java.
Enter the following code.

```
public class FirstProgram {
public static void main(String[] args) {
System.out.println("Welcome to COS 201!");
System.out.println("I am learning computer programming.");
System.out.println("Programming begins with problem-solving.");
}
}
```

Save the file using Ctrl + S.

Step 3 — Understand the code
The filename must be FirstProgram.java because the public class is named FirstProgram. Java distinguishes between uppercase and lowercase letters.

Step 4 — Compile and run the programme
In VS Code, select Terminal → New Terminal.
If your terminal opens in the COS201-Programming-I folder, enter:

```
cd week01
javac FirstProgram.java
java FirstProgram
```

If your terminal is already inside week01, omit the cd week01 command.
Expected output:

```
Welcome to COS 201!
I am learning computer programming.
Programming begins with problem-solving.
```

If the programme compiles successfully, Java creates a FirstProgram.class file containing bytecode. Running java FirstProgram executes the compiled class.
If you encounter an error:
If javac is not recognised, check whether the JDK is installed and correctly configured.
If the compiler reports a class-name error, check the filename and public class name.
If you receive a missing semicolon error, inspect the output statements.

If the programme runs but displays unexpected text, check the strings inside quotation marks.

Step 5 — Modify the programme
Add two additional output statements:
System.out.println("My programme is running successfully.");
System.out.println("I will practise programming every week.");
Save the file, compile it again and run it.
Record what changed in the output. This exercise demonstrates that source code can be modified and re-executed.

Step 6 — Complete the Week 1 practical task
Modify FirstProgram.java so that it displays the following information:
The course code and title.
Your programme of study.
Three reasons programming is important to your discipline.
One personal learning goal for COS 201.
Use your own wording and avoid including unnecessary personal information in the source code.
Example output format:
Course: COS 201 - Computer Programming I
Programme: Computer Science
Reason 1: Programming helps solve computational problems.
Reason 2: Programming supports automation.
Reason 3: Programming enables software development.
My goal: I want to develop simple working applications.
This is an illustrative output; students should supply their own programme and learning goal.

14. Step-by-step guide: Submit your practical through GitHub

The submission process has two parts: saving your code in a GitHub repository and sending the lecturer the repository URL through https://docs.google.com/forms/d/e/1FAIpQLScfFS-TTZzdgAZ9qPEFp8gNJpHCu0cYudvHJizNCbKOACAb1A/viewform?usp=header

Step 1 — Create or access your GitHub account
Visit GitHub.
Sign in or create an account if you do not already have one.
Use your institution-approved or personal email where appropriate.
Verify your email address if requested.

Step 2 — Create the repository
On GitHub:
Select the + menu and choose New repository.
Enter the repository name:
COS201-Programming-I

3. Add the description:

COS 201 Week 1 practical exercises and programming assignments.
Choose Public repository.
Select Add a README file.
Click Create repository.
The repository is your online project space. It stores your source code, documentation and the history of changes you commit.

Step 3 — Clone the repository using GitHub Desktop
A. Open GitHub Desktop
Sign in with the GitHub account associated with your repository.

B. Clone the repository
Select File → Clone Repository, choose the repository you created and specify a local folder where you want the project saved.
C. Work inside the cloned folder
Open the local repository in VS Code. Create the week01 folder and FirstProgram.java file inside it. Do not create the project in an unrelated folder, or GitHub Desktop may not detect the changes.
Your local project should now resemble this:

```
COS201-Programming-I/
├── README.md
└── week01/
└── FirstProgram.java
```

Step 4 — Create the README file
Open README.md and add a short description of your work.

```
# COS 201 - Computer Programming I
## Week 1: Introduction to Programming and Problem-Solving
### Student Details
- Name: [Your name]
- Programme: [CSC/CYB/INS/SEN/ICH]
- Course Code: COS 201
### Practical Exercise
Created and executed my first Java programme.
### File
- week01/FirstProgram.java
### Skills Practised
- Java class and main method
- Output statements
- Compilation and execution
- GitHub version control
```

Do not include your password, access tokens or other confidential information.

Step 5 — Commit your changes
Open GitHub Desktop.
Select the Changes tab.
Review the files listed as modified or newly created.
Confirm that README.md and week01/FirstProgram.java appear.
Enter the commit summary:
Complete Week 1 Java practical
Select Commit to main, or commit to the current branch if the repository uses a different branch name.
A commit saves a snapshot of your changes in the repository's local history. It does not automatically upload those changes to GitHub.

Step 6 — Push your changes to GitHub
After committing:
Click Push origin in GitHub Desktop.
Wait for the operation to finish.
Open your repository on the GitHub website.
Refresh the page.
Confirm that README.md and week01/FirstProgram.java are visible.
Your work is not fully submitted online until the changes have been pushed successfully.

Step 7 — Copy the repository link
On your repository page, copy the URL from the browser's address bar.
It will resemble this format:

https://github.com/your-username/COS201-Programming-I
The example is illustrative. Submit the actual URL of your own repository, not the example.
If the repository is private, ensure that the lecturer has the necessary access. A private repository link alone does not grant access.

Step 8 — Submit the link through https://docs.google.com/forms/d/e/1FAIpQLScfFS-TTZzdgAZ9qPEFp8gNJpHCu0cYudvHJizNCbKOACAb1A/viewform?usp=header
Your submission should contain:
Your full name.
Your deparment.
The assignment title: COS 201 Week 1 Practical.
The actual GitHub repository URL.

Student submission checklist
Before submitting make sure;
The Java programme compiles and runs correctly.
The source file is saved as week01/FirstProgram.java.
The README.md describes the Week 1 practical.
The changes have been committed in GitHub Desktop.
The latest commit has been pushed to GitHub.
The lecturer can access the repository.
The actual repository URL has been submitted through the link provided.

15. Week 1 assignments, to be written on paper for submission.

Activity A — Identify the IPO components
For each problem, write the input, process and output.

1. Calculate the area of a rectangle with length 12 and width 5.
2. Calculate the average of four examination scores.
3. Determine whether a number is positive, negative or zero.
4. Calculate the total price of six items costing ₦250 each.
5. Calculate the number of failed login attempts recorded in a sample security log.

Activity B — Write algorithms

1. Write an algorithm for each task.
2. Accept two numbers and display their sum.
3. Calculate the area of a circle using $A=pi\ r^2$.
4. Accept a student's score and determine whether the student passed, using 50 as the pass mark.
5. Accept three numbers and display the largest.
6. Calculate the total cost of items purchased, given quantity and unit price.

For each algorithm, identify its input, processing steps and output.

Activity C — Pseudocode and flowcharts
Choose any two problems from Activity B.
For each selected problem:
Write the algorithm in numbered steps.
Convert the algorithm into pseudocode.
Draw a flowchart using the correct symbols.

Week 2: Development Environment and First Java Program

1. Introduction

In Week 1, we studied programming, problem-solving, algorithms, pseudocode, flowcharts and the stages involved in developing a computer program. In this week, we will move from the theoretical foundation to the practical environment where programs are written, compiled, executed, tested and submitted.
Before a programmer can write and run Java code, the computer must have the appropriate development tools installed and configured.
By the end of this lecture, students should be able to install or verify the required software, create a Java project in Visual Studio Code, write and execute their first Java program, identify common programming errors, and submit their work through GitHub.

2. Learning objectives

At the end of this lecture, students should be able to:
Explain the meaning of a development environment and an IDE.
Differentiate between the JDK, JVM and Java compiler.
Install and verify the Java Development Kit (JDK).
Install and configure Visual Studio Code for Java programming.
Create, compile and run a Java program.
Explain the structure of a basic Java program.
Identify and correct common Java errors.
Explain version control, Git, GitHub and GitHub Desktop.
Create a GitHub repository and submit a working Java program.
Follow a clear, verifiable submission procedure for coursework.

Part A — Understanding the Development Environment

3. What is a development environment?

A development environment is a collection of software tools that enables a programmer to write, test, run, debug and maintain computer programs.
For this course, our development environment will contain the following:
Java Development Kit (JDK)
Supplies the Java compiler and other tools required to develop Java programs.
Visual Studio Code (VS Code)
A code editor that helps us write, organize and work with Java source files.
Git
Tracks changes made to source code over time.
GitHub Desktop
Provides a graphical interface for committing changes and sending them to a GitHub repository.
GitHub
Hosts repositories online so students and lecturers can access, review and manage submitted code.
These tools have different functions. Installing VS Code alone does not necessarily install Java, and creating a GitHub account does not automatically configure Git on a computer.

4. What is an IDE?

An Integrated Development Environment (IDE) combines several programming tools in one application.
Typical features include a code editor, compiler integration, debugging tools and project management.
VS Code is primarily a code editor, but extensions can provide many Java development features. Other examples of Java IDEs include IntelliJ IDEA and Eclipse.
For COS 201, we will use VS Code because it is lightweight, widely used and supports Java development through extensions.

5. Understanding the JDK, JVM and compiler

These three terms are important for understanding how Java programs work.
The relationship can be represented as follows:
Java source code

HelloWorld.java
Java compiler
javac HelloWorld.java
Java bytecode
HelloWorld.class
Java Virtual Machine (JVM)
Executes the bytecode
Program output
Displayed on the screen
Example: A Java program is saved in HelloWorld.java. The compiler produces HelloWorld.class, and the JVM executes the bytecode. The .java file is the human-readable source code that you will submit for this course.

Part B — Installing and Configuring the Software

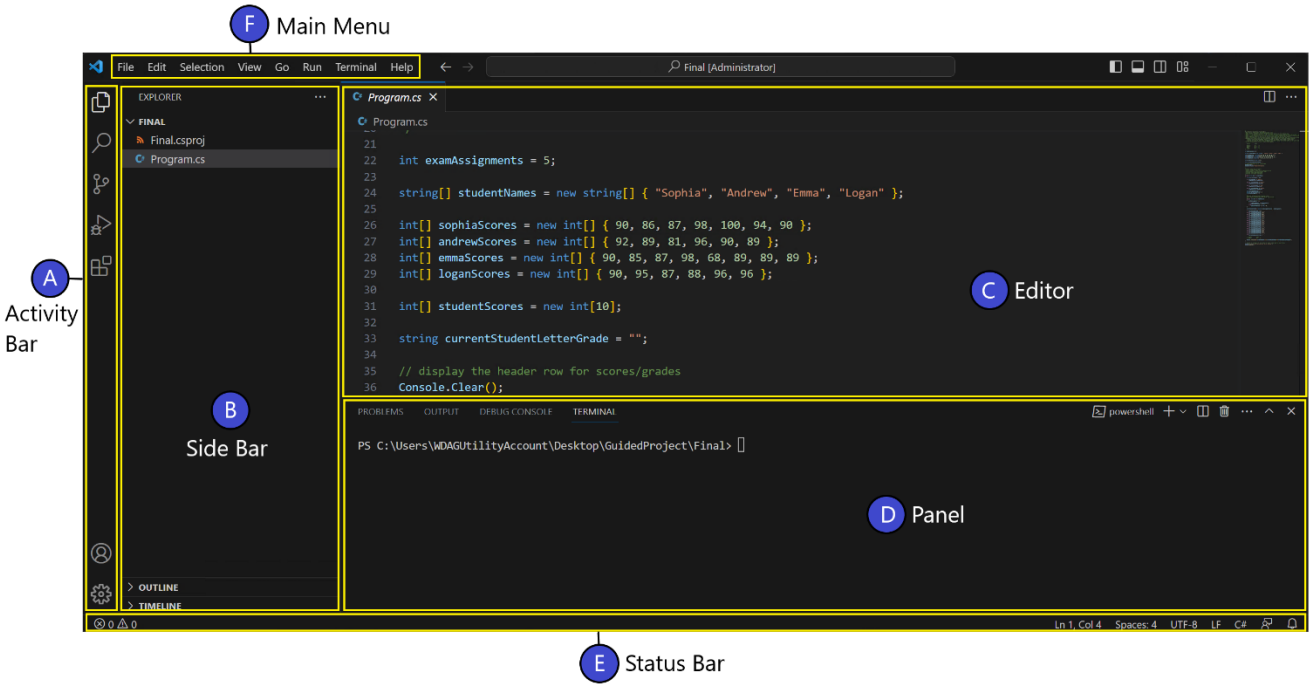
6. Step 1: Install or verify the JDK

You need a JDK before you can compile Java source code.
Use an official download source:
Download Java from Oracle
Download OpenJDK builds from Adoptium
Installation procedure on Windows
Open one of the official links mentioned above.
Select the Windows download that matches your computer's architecture, usually x64 for a standard Intel/AMD Windows PC.
Download the JDK installer.
Run the installer.
Follow the installation prompts.
If the installer offers options to configure the PATH or JAVA_HOME environment variable, enable them if appropriate.
Complete the installation.
Verify the installation
Open Command Prompt or the VS Code terminal.
Run:
java -version
Then run:
javac -version
A successful installation should display version information for both commands. The exact version numbers depend on the JDK you installed.
For example, your output may resemble:
java version "21..."
javac 21...
The dots represent additional version information; your computer's output may differ.
What if the commands are not recognized?
Close and reopen the terminal after installation.
Check whether the JDK installation completed successfully.
Check that the JDK's bin directory is available through the system PATH.
If necessary, ask your lab instructor or administrator to help configure the environment variables.
Do not assume that Java is installed simply because a Java-related application appears in the Start menu.

7. Step 2: Install Visual Studio Code

Download VS Code from its official website:
Visual Studio Code — Download
Follow these steps:
Select the correct download for your operating system.
Run the installer.
Follow the installation instructions.

Launch Visual Studio Code.



A - The **Activity Bar** is the narrow vertical bar located on the far left-hand side of the window. The Activity bar lets you switch between views (such as Explorer or Extensions) and gives you other context-specific indicators.
B - The **Side Bar** contains views that provide tools and resources. The Side Bar views (such as EXPLORER) are very helpful when you're working on a code project.
C - The **Editor** is the area used to edit your files. You can open as many editors as you like side by side vertically and horizontally.
D - The **Panel** area is used to display different panels below the editor region for output or debug information, errors and warnings, or an integrated terminal.
E - The **Status Bar** is the horizontal bar along the bottom of the window that displays information about the opened project and the files you edit.
F - The **Main Menu** is the menu interface at the top of the application window. The menu options enable you to save, edit, and run your code (and much more).

8. Step 3: Install Java support in VS Code

Open VS Code.
Select the Extensions icon on the left, or press Ctrl + Shift + X.
Search for Extension Pack for Java.
Select the official Microsoft extension pack.
Click Install.
Allow the installation to complete.
If requested, reload VS Code.
The extension pack adds Java language support and useful development features. It does not replace the JDK, so keep the JDK installed separately.

9. Step 4: Install Git

Git is a version-control system that records changes to files and supports collaboration.
Download Git from:
Official Git website
For Windows:
Download the Windows installer.
Run the installer.
Follow the prompts, using the default settings unless your instructor specifies otherwise.

Complete the installation.
Open a new terminal.
Run:
git --version
If Git is installed correctly, the terminal displays a Git version number.

10. Step 5: Install GitHub Desktop

GitHub Desktop makes common Git tasks easier for beginners.
Download it from:
GitHub Desktop — Official download
Procedure:
Download and install GitHub Desktop.
Open the application.
Sign in to your GitHub account, or create an account at GitHub.
Complete the authentication process.
Confirm that the application shows your account.
Account safety: Never share your password, authentication code or private access token with another student or include it in a Java source file or README.

Part C — Creating and Running Your First Java Program

11. Create a folder for COS 201

Organizing files correctly is an important programming skill. A well-organized project is easier to test, review, submit and maintain.
Create a main folder called:
COS201-Programming-I
Inside it, create a folder called week02.
Your folder structure should initially look like this:

```
COS201-Programming-I/
└── week02/
```

How to create the folders
Open File Explorer on Windows.
Navigate to a suitable location, such as Documents.
Create the folder COS201-Programming-I.
Open it and create a folder named week02.
Open VS Code.
Select File → Open Folder.
Select the COS201-Programming-I folder.
If VS Code asks whether you trust the folder, confirm only if it is a folder you created or otherwise trust.

12. Write your first Java program

Inside VS Code:
In the Explorer panel, select the week02 folder.
Click the New File icon.
Enter the exact filename HelloWorld.java.
Type the following code.

```
public class HelloWorld {
public static void main(String[] args) {
System.out.println("Hello, World!");
System.out.println("Welcome to COS 201.");
System.out.println("I am learning Java programming.");
}
}
```

5. Save the file using Ctrl + S.

Important: Java is case-sensitive. The filename must be HelloWorld.java because the public class is named HelloWorld. Do not save the file as HelloWorld.java.txt.

13. Understand every part of the program

Consider the first line:
public class HelloWorld {
public is an access modifier.
class declares a Java class.
HelloWorld is the class name.
{ opens the class body.
Now consider the main method:
public static void main(String[] args) {
For now, understand that this is the standard entry point used when launching a simple Java application.
public makes the method accessible to the Java launcher.
static allows the method to be invoked without first creating an object of the class.
void means the method does not return a value.
main is the method name recognized as the application's entry point.
String[] args allows command-line arguments to be passed to the program.
{ opens the method body.
The following statement displays text:
System.out.println("Hello, World!");
System.out refers to the standard output stream.
println() prints the supplied value and then moves to a new line.
"Hello, World!" is a string literal.
; marks the end of the statement.
Finally, the closing braces } mark the end of the method and the class.
Expected output
When the program runs successfully, the terminal should display:
Hello, World!
Welcome to COS 201.
I am learning Java programming.
The output should appear on three separate lines because the program contains three println() statements.

14. Compile and run the program using the terminal

Although VS Code may offer a Run button, it is important to understand how to compile and run Java programs directly.
Step 1: Open the terminal
In VS Code, select Terminal → New Terminal.
Ensure the terminal is positioned in the COS201-Programming-I folder. If the terminal is already inside week02, skip the directory-change command below.
Step 2: Enter the week02 folder
If you are currently in the main project folder, type:
cd week02
Step 3: Compile the program
Enter:
javac HelloWorld.java
If the program has no compilation errors, the command should normally finish without displaying an error message. It will create a file named HelloWorld.class in the current directory.
Step 4: Run the program
Enter:
java HelloWorld
Notice that you do not add .java or .class after the class name when using this command.
Step 5: Check the output

Confirm that the three expected lines are displayed.
Your folder may now contain:
week02/
├── HelloWorld.java
└── HelloWorld.class
The .java file contains your source code. The .class file contains compiled bytecode. For this coursework, submit the source code and the required documentation; do not rely on submitting only the compiled .class file.

15. Alternative: Run Java from VS Code

Once the JDK and Java extension pack are configured:
Open HelloWorld.java.
Look for the Run option above the main method, or use the Java run controls provided by VS Code.
Select Run.
Examine the output in the terminal or the designated output panel.
If the Run option is missing, confirm that the Java extension pack is installed, the JDK is detected, and the file is saved with the .java extension.
The terminal method remains useful because it helps you understand compilation and execution independently of the editor.

Part D — Improving the First Program

16. Practical example: Personalize your program

Modify your program to display your course and academic information. Replace the sample details with your own information.

```
public class HelloWorld {
public static void main(String[] args) {
System.out.println("COURSE: COS 201");
System.out.println("COURSE TITLE: Computer Programming I");
System.out.println("PROGRAMME: Your Programme");
System.out.println("WEEK: 2");
System.out.println("TOPIC: Development Environment and Java");
System.out.println("GOAL: To learn how to write Java programs.");
}
}
```

Save the file, compile it and run it again:

```
javac HelloWorld.java
java HelloWorld
```

Expected learning outcome: You should be able to modify a Java program, save the source file, compile it and verify that the output matches your instructions.

17. Using print() and println()

Java provides different methods for displaying information.
The println() method displays information and moves to a new line:

```
System.out.println("Computer");
System.out.println("Programming");
```

Output:

```
Computer
Programming
```

The print() method displays information without automatically moving to the next line:

```
System.out.print("Computer ");
System.out.print("Programming");
```

Output:

```
Computer Programming
```

Practice exercise

Predict the output before running this program:

```
public class PrintExample {
public static void main(String[] args) {
System.out.print("COS ");
System.out.println("201");
System.out.println("Java");
}
}
```

Then save it as PrintExample.java, compile it with javac PrintExample.java, and run it with java PrintExample. Remember that a public class named PrintExample must be saved in a file named PrintExample.java.

18. Using comments in Java

Comments explain code to people reading it. The Java compiler ignores ordinary comments when compiling the program.

Single-line comment

```
// This line explains the purpose of the next statement.
System.out.println("Welcome to COS 201.");
```

Multi-line comment

```
/*
This program introduces students
to basic Java output statements.
*/
```

Comments are useful when explaining the purpose of a program, describing a complex section of code, or identifying an assignment. They should be meaningful rather than used to hide errors or replace proper explanations.

19. Common errors and how to correct them

Beginning programmers often encounter errors. An error is not proof that you cannot program; it is a signal to inspect your code or environment.

Exercise: Find and fix the error

The following code contains an error:

```
public class ErrorExample {
public static void main(String[] args) {
System.out.println("Learning Java")
}
}
```

Your task: Identify the missing symbol, correct the program, save it as ErrorExample.java, compile it and run it.

Corrected version:

```
public class ErrorExample {
public static void main(String[] args) {
System.out.println("Learning Java");
}
}
```

The missing semicolon was the problem.

A systematic debugging method

Read the error message carefully.

Identify the file and line mentioned in the message.

Inspect the relevant line and nearby code.

Correct one problem at a time.

Save the file.

Compile again.

Run the program and verify the result.
Do not repeatedly change unrelated parts of a program without checking what caused the original error.

Part E — Version Control and GitHub

20. What is version control?

Version control is a system for recording changes to files over time. It helps programmers identify what changed, restore earlier versions and collaborate with other developers.
For example, suppose you wrote a Java program on Monday and improved it on Wednesday. Version control can record both stages so that you can inspect the differences between them.

21. Understand Git, GitHub and GitHub Desktop

These terms are related, but they do not mean the same thing. For example, saving a file in VS Code does not automatically commit or push it to GitHub.

22. Create your COS 201 repository on GitHub

We will use one repository for the course and keep each week's practical work in its own folder.
Step 1: Sign in to GitHub
Open GitHub and sign in. If you do not have an account, create one using the platform's instructions.
Step 2: Create a repository
Select the option to create a new repository.
Enter this repository name:
COS201-Programming-I
Enter a description such as:
Practical exercises for COS 201 Computer Programming I.
Choose Private unless your lecturer requires a public repository.
Select the option to add a README file, if offered.
Create the repository.
Privacy note: If you choose a private repository, your lecturer must be granted access to it through the appropriate GitHub permissions. Sending a private repository's URL alone does not grant access.

23. Clone the repository using GitHub Desktop

Cloning creates a local copy of your online repository on your computer.
Open GitHub Desktop.
Sign in if necessary.
Select File → Clone Repository.
Select your COS201-Programming-I repository from the GitHub.com tab.
Choose a local folder where you want to store the project.
Select Clone.
After cloning, select Show in Explorer or the equivalent option to open the local project folder.
Open the local folder in VS Code.
Your project should now have a structure similar to:

```
COS201-Programming-I/
├── README.md
└── week02/
```

Create or copy HelloWorld.java into the week02 folder.
Important: Always work inside the cloned repository folder. If you edit a separate copy elsewhere on your computer, GitHub Desktop may not detect those changes in your repository.

24. Prepare your README documentation

A README file explains what the project contains and how it should be used. It is usually written in Markdown and saved as README.md.
Open the repository's README.md in VS Code and use this template:

```
# COS 201 — Computer Programming I
## Student Information
```

- Name: Enter your full name
- Matriculation Number: Enter your matriculation number
- Programme: Enter your programme
- Course: COS 201
- Week: 2
Practical Title
Development Environment and First Java Program
Objective
To configure the Java development environment, write a simple Java program, compile it, execute it and submit the source code through GitHub.
Tools Used
- Java Development Kit (JDK)
- Visual Studio Code
- Git
- GitHub Desktop
- GitHub
Files
- `week02/HelloWorld.java`: Java source code for the practical.
How to Run

1. Open a terminal in the `week02` directory.

2. Compile the program using `javac HelloWorld.java`.

3. Run the program using `java HelloWorld`.
Learning Outcome
I practised creating, compiling and running a Java program and submitting source code through GitHub using https://docs.google.com/forms/d/e/1FAIpQLScfFS-TTZzdgAZ9qPEFp8gNJpHCu0cYudvHJizNCbKOACAb1A/viewform?usp=header
Replace the placeholder information with your actual details before submission. Do not put passwords, authentication tokens or other confidential credentials in the README.

25. Commit your work using GitHub Desktop
A commit records a set of changes in your local Git repository.
Follow these steps:
Save HelloWorld.java in the week02 folder.
Save your README.md.
Open GitHub Desktop.
Confirm that the correct repository is selected.
Open the Changes tab.
Review the files listed as changed or added.
Select the files you intend to include.
In the summary field, enter:
Complete Week 2 Java practical
Select Commit to main or the equivalent current-branch commit option.
Wait for the commit to complete.
A commit is a local record until you push it to GitHub. If you stop here, your lecturer may not see the latest changes online.

26. Push your work to GitHub
After committing:
In GitHub Desktop, select Push origin. Depending on the application's state, the button or menu wording may differ.
Wait for the push operation to complete.
Open your repository on GitHub in a web browser.

Refresh the page if necessary.
Confirm that README.md and week02/HelloWorld.java are visible online.
Open the Java file in the browser and inspect its contents.
Your online repository should resemble:

```
COS201-Programming-I/
├── README.md
└── week02/
└── HelloWorld.java
```

You may also see other files or folders if you have completed additional exercises.

27. How to obtain your repository link

Open the repository in your browser and copy its URL from the address bar.
A repository URL generally looks like this:
https://github.com/YOUR-USERNAME/COS201-Programming-I
This is an example format, not your actual repository address. Replace it by copying the real URL from your own GitHub account.
For a private repository, make sure the lecturer has the appropriate access before submitting the link.

Part F — Submitting Your Practical Work to the Lecturer

28. The complete submission workflow

The following sequence is the recommended workflow for every practical assignment in COS 201.

1. Write the code

Create and save your Java source file in VS Code.

2. Compile and run

Use javac and java to confirm that the program works.

3. Document the work

Update README.md with your details and practical objective.

4. Commit changes

Record the files and changes in GitHub Desktop.

5. Push to GitHub

Upload your committed work to the online repository.

6. Verify and submit

Check the online files and send the repository URL through https://docs.google.com/forms/d/e/1FAIpQLScfFS-TTZzdgAZ9qPEFp8gNJpHCu0cYudvHJizNCbKOACAb1A/viewform?usp=header

29. What exactly should you submit?

For this Week 2 practical, submit the following:
COS 201 — Week 2 Practical Submission
Dear Lecturer,
I am submitting my Week 2 practical assignment for COS 201 — Computer Programming I.
Student Name: [Your Full Name]
Matriculation Number: [Your Matric Number]
Programme: [Your Programme]
Assignment: Development Environment and First Java Program
GitHub Repository: [Paste your actual repository URL here]
I have included my Java source code and README documentation in the repository. I have also compiled and executed the program to verify the output.
Thank you.
Before sending the message, replace every placeholder with the correct information. Do not submit the illustrative URL from this lecture.

Week 3: Variables, Data Types, Expressions and Input/Output

1. Introduction

In Week 1, we learned how to approach a problem using algorithms, pseudocode and flowcharts. In Week 2, we installed the development tools, wrote our first Java program, compiled and executed it, and learned how to submit our work through GitHub.
This week, we will learn how to make our programs work with data.
Most useful programs must store information, perform calculations, receive input from users and display results.
For example:
A student-result program stores student names and scores and calculates averages.
A cybersecurity program stores the number of login attempts and evaluates a threshold.
An information system stores quantities, prices and account-status information.
A software application processes text, numbers and user selections.
To perform these tasks, programmers use variables, data types, expressions and input/output statements.

2. Learning objectives

By the end of this lecture, students should be able to:
Define variables, constants and data types.
Declare, initialize and update Java variables.
Identify and use Java's primitive data types.
Distinguish between integers, floating-point numbers, characters, strings and Boolean values.
Apply Java naming conventions and rules for identifiers.
Construct arithmetic, relational and logical expressions.
Understand operator precedence and type conversion.
Receive user input using the Scanner class.
Display formatted results using System.out.print(), println() and printf().
Develop, test, document and submit a Java program using GitHub.

Part A — Variables and Constants

3. What is a variable?

A variable is a named storage location used by a program to hold a value that can be accessed and, when appropriate, changed during execution.
Imagine a labelled container. The label tells you what information it represents, and the contents represent its current value.
For example:
int age = 20;
This statement declares a variable named age, specifies that it stores an integer, and initializes it with the value 20.
Illustration of a variable
age
20
Variable name and current value

Other examples include:
int numberOfStudents = 45;
double averageScore = 76.5;
String studentName = "Amina";
boolean isRegistered = true;
Each variable has a name, a data type and a value.

4. Declaring and initializing variables

These are two related but different operations.

4.1 Variable declaration
A declaration specifies a variable's data type and name.
int score;
This declares an integer variable called score, but it does not explicitly assign an initial value.

4.2 Variable initialization
Initialization assigns the first value to a variable.
int score = 85;
The variable is declared and initialized in one statement.

4.3 Assignment
Assignment gives a variable a value, or changes its existing value.
int score = 85;
score = 90;
After the second statement, the value of score is 90.

4.4 Updating a variable
int total = 10;
total = total + 5;
System.out.println(total);
Output:
15
The expression total + 5 is evaluated using the current value of total, and the result is assigned back to total.
Java also supports a shorter form:
total += 5;
This adds 5 to the current value of total.

5. Rules for naming variables
A variable name is also called an identifier.

Java identifiers follow these rules:
They may contain letters, digits, underscores (_) and dollar signs ($).
They cannot begin with a digit.
They cannot contain spaces.
They cannot be Java reserved keywords such as class, int or public.
Java is case-sensitive: score, Score and SCORE are different identifiers.

Examples:
Recommended naming convention
Use camelCase for ordinary variable names: the first word begins with a lowercase letter, and subsequent words begin with uppercase letters.

Examples:
int studentAge;
double examinationScore;
String courseTitle;
Descriptive names make programs easier to understand and maintain.

6. What is a constant?
A constant is a value that should not be reassigned after initialization. In Java, the final keyword can be used to declare a variable that may be assigned only once.
Example:
final double TAX_RATE = 0.075;

The TAX_RATE variable cannot be reassigned after it has been initialized.
For example, this would produce a compilation error:
final int MAX_ATTEMPTS = 3;
MAX_ATTEMPTS = 5;
The conventional naming style for constants is uppercase words separated by underscores.
Constants are useful for values that should remain fixed throughout a calculation, such as a conversion factor or a defined maximum.

Part B — Java Data Types

7. What is a data type?

A data type defines the kind of value a variable can represent and the operations that are available for it.
For example, a student's age is typically represented by an integer, while an average examination score may require a decimal value.
Java is a statically typed language: a variable's declared type determines what values and operations are permitted, subject to Java's type-conversion rules.

8. Java's primitive data types

Java has eight primitive data types.
For introductory programming, students will use int, double, char and boolean frequently.
Important points
Use int for ordinary whole-number calculations.
Use double for most introductory decimal calculations.
Use the suffix L for a long literal when needed.
Use the suffix f or F for a float literal.
A char literal uses single quotation marks, such as 'A'.
A Boolean literal is true or false, without quotation marks.
For example:
int age = 21;
double score = 82.5;
char grade = 'B';
boolean isPresent = true;

9. Difference between char and String

A char represents one UTF-16 code unit and is written with single quotation marks.
char grade = 'A';
A String represents a sequence of characters and is written with double quotation marks.
String studentName = "Amina Bello";
String is a Java class, not one of the eight primitive types.
Compare the following:
char letter = 'A';
String word = "A";
String message = "Welcome to COS 201";
Even though the first two variables may appear similar, they have different types.

10. The String data type

A string is used to store text.
Examples:
String firstName = "Amina";
String courseCode = "COS 201";
String programme = "Computer Science";
You can combine strings using the + operator:
String firstName = "Amina";

```
String lastName = "Bello";
String fullName = firstName + " " + lastName;
System.out.println(fullName);
```
Output:
Amina Bello
When a string is combined with another value using +, Java may convert the other value to text.
```
int age = 20;
System.out.println("Age: " + age);
```
Output:
Age: 20
This is called string concatenation.

Part C — Expressions and Operators

12. What is an expression?
An expression is a combination of values, variables, operators and sometimes method calls that Java evaluates to produce a result.
Examples:
```
10 + 5
score * 2
averageScore >= 50
```
The first expression produces 15. The second multiplies the value of score by two. The third evaluates to a Boolean value, either true or false.

13. Arithmetic operators
Arithmetic operators perform mathematical calculations.
Example 1: Calculating a total
```
int mathematics = 70;
int programming = 80;
int total = mathematics + programming;
System.out.println(total);
```
Output:
150

Example 2: Calculating an average
```
double score1 = 70;
double score2 = 80;
double score3 = 90;
double average = (score1 + score2 + score3) / 3.0;
System.out.println(average);
```
Output:
80.0
The parentheses ensure that the three scores are added before division.

Example 3: Integer division
Consider:
```
int result = 5 / 2;
System.out.println(result);
```
Output:
2

Both operands are integers, so Java performs integer division and discards the fractional part of the result.
To obtain a decimal result, use a floating-point operand:
```
double result = 5.0 / 2;
```

System.out.println(result);
Output:
2.5
Important: Assigning an integer-division result to a double does not restore the discarded fraction. The division itself must use floating-point arithmetic when a fractional result is required.
Example 4: The remainder operator
System.out.println(10 % 3);
System.out.println(12 % 2);
Output:
1
0
The remainder operator is useful for determining whether an integer is even or odd.
int number = 8;
System.out.println(number % 2 == 0);
Output:
True

14. Assignment operators

Assignment operators store or update values.
Example:
int balance = 100;
balance += 25;
balance -= 10;
System.out.println(balance);
Output:
115
The value begins at 100, increases to 125, and then decreases to 115.

15. Relational operators

Relational operators compare values and produce a Boolean result.
Example:
int score = 75;
System.out.println(score >= 50);
System.out.println(score == 100);
System.out.println(score < 40);
Output:
true
false
false
A common beginner mistake is confusing assignment with comparison:
score = 50; // Assigns 50 to score.
score == 50; // Evaluates whether score equals 50.
The equality expression can be used in a condition or assigned to a Boolean variable.
boolean hasPassed = score >= 50;

16. Logical operators

Logical operators combine or reverse Boolean conditions.

Example: Checking a valid score

Operator	**Meaning**	**Example**
&&	Logical AND	age >= 18 && age <= 25
||	Logical OR	score < 0 || score > 100

!	Logical NOT	!isRegistered

Suppose a score must be between 0 and 100, inclusive.
int score = 75;
boolean validScore = score >= 0 && score <= 100;
System.out.println(validScore);
Output:
true
The && operator means both conditions must be true.
If the score were 105, the second condition would be false, so the combined result would be false.

Example: Checking an invalid score
int score = 105;
boolean invalidScore = score < 0 || score > 100;
System.out.println(invalidScore);
Output:
true
The || operator returns true if at least one condition is true.

17. Operator precedence

When an expression contains several operators, Java follows rules that determine which operations are performed first.
Consider:
int result = 2 + 3 * 4;
System.out.println(result);
Output:
14
Multiplication is performed before addition.
If you want addition to happen first, use parentheses:
int result = (2 + 3) * 4;
System.out.println(result);
Output:
20
For beginners, the safest practice is to use parentheses whenever they make the intended order of calculation clearer.

18. Type conversion and casting

Java sometimes converts a value from one numeric type to another.

18.1 Widening conversion

A widening conversion moves from a type that can represent a smaller range or precision to a wider type, where the conversion is permitted by Java.
int number = 25;
double decimalNumber = number;
System.out.println(decimalNumber);
Output:
25.0

18.2 Narrowing conversion

A narrowing conversion may lose information and generally requires an explicit cast.
double value = 9.8;
int wholeNumber = (int) value;
System.out.println(wholeNumber);
Output:

9
Casting the double to an int discards the fractional part; it does not round to the nearest whole number.

18.3 Converting text into a number
Values read from the keyboard often arrive as text when using Scanner.nextLine(). Java provides parsing methods to convert suitable strings to numbers.

```
String text = "42";
int number = Integer.parseInt(text);
System.out.println(number + 8);
```

Output:
50
If the string does not contain a valid integer representation, Integer.parseInt() throws a NumberFormatException. We will explore handling input errors further in later weeks.

Part D — Input and Output in Java

19. What is input?
Input is information supplied to a program for processing.
Input can come from a keyboard, a file, a database, a sensor or another program. For this week, we will focus on keyboard input.
Examples include:
A student's name.
Three examination scores.
The quantity and price of an item.
The number of security incidents recorded.
A user's choice from a menu.

20. What is output?
Output is information produced by a program.
For our practical exercises, output will generally be displayed in the terminal.
Java provides several methods for displaying information.
System.out.print()
Displays information without automatically starting a new line.

```
System.out.print("Hello ");
System.out.print("COS 201");
```

Output:
Hello COS 201
System.out.println()
Displays information and moves to a new line.

```
System.out.println("Hello");
System.out.println("COS 201");
```

Output:
Hello
COS 201
System.out.printf()
Displays formatted output.

```
String name = "Amina";
double score = 82.567;
System.out.printf("Name: %s%n", name);
System.out.printf("Score: %.2f%n", score);
```

Output:
Name: Amina
Score: 82.57

Here:
%s formats a string.
%d formats an integer.
%.2f formats a floating-point value to two digits after the decimal point.
%n produces a platform-appropriate line separator.

21. Reading keyboard input using Scanner

The Scanner class is one of the simplest ways to read keyboard input in introductory Java programs.
First, import the class:
import java.util.Scanner;
Then create a scanner:
Scanner input = new Scanner(System.in);
This creates a Scanner object named input that reads from the standard input stream, normally the keyboard.
Common methods include:

Example: Reading a student's name

```
import java.util.Scanner;
public class StudentInput {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = input.nextLine();
System.out.println("Welcome, " + name + "!");
}
}
```

If the user enters Amina Bello, the output will be:
Enter your name: Amina Bello
Welcome, Amina Bello!
The name is read as a complete line, including the space between the first and last names.

22. Reading integers and decimal numbers

Example: Reading an age

```
import java.util.Scanner;
public class AgeInput {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = input.nextInt();
System.out.println("Your age is " + age);
}
}
```

When prompted, the user enters a whole number.

Example: Reading a price

```
import java.util.Scanner;
public class PriceInput {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter item price: ");
double price = input.nextDouble();
System.out.printf("Item price: %.2f%n", price);
```

```
}
}
```

The program reads a decimal number and displays it to two decimal places.
Note: The program expects valid input. If a user enters text where an integer is expected, methods such as nextInt() can throw an InputMismatchException. Robust input validation will be developed further in subsequent lessons.

23. A common Scanner problem: nextInt() followed by nextLine()

Consider this code:

```
int age = input.nextInt();
String name = input.nextLine();
```

After nextInt() reads the integer, the line separator entered by the user may remain in the input buffer. The subsequent nextLine() can consume the remainder of that line instead of waiting for a name.

A common solution is:

```
int age = input.nextInt();
input.nextLine(); // Consume the remaining line separator.
System.out.print("Enter your name: ");
String name = input.nextLine();
```

24. Example: Collecting student information

```
import java.util.Scanner;
public class StudentDetails {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = input.nextLine();
System.out.print("Enter your programme: ");
String programme = input.nextLine();
System.out.print("Enter your level: ");
int level = input.nextInt();
System.out.println("\n--- STUDENT DETAILS ---");
System.out.println("Name: " + name);
System.out.println("Programme: " + programme);
System.out.println("Level: " + level);
}
}
```

Sample interaction

```
Enter your name: Amina Bello
Enter your programme: Computer Science
Enter your level: 200
--- STUDENT DETAILS ---
Name: Amina Bello
Programme: Computer Science
Level: 200
```

This example combines three concepts:
Variables store the information.
Data types determine the kind of values stored.
Input and output statements allow interaction with the user.

Part E — Complete Practical Programs

25. Practical 1: Simple calculator

This program reads two numbers and calculates their sum, difference, product and quotient.

```
import java.util.Scanner;
public class SimpleCalculator {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter the first number: ");
double firstNumber = input.nextDouble();
System.out.print("Enter the second number: ");
double secondNumber = input.nextDouble();
double sum = firstNumber + secondNumber;
double difference = firstNumber - secondNumber;
double product = firstNumber * secondNumber;
System.out.printf("Sum: %.2f%n", sum);
System.out.printf("Difference: %.2f%n", difference);
System.out.printf("Product: %.2f%n", product);
if (secondNumber != 0) {
double quotient = firstNumber / secondNumber;
System.out.printf("Quotient: %.2f%n", quotient);
} else {
System.out.println("Division is undefined for a zero divisor.");
}
}
}
```

Step-by-step explanation

Import Scanner so the program can read keyboard input.
Create the input object.
Read the first and second numbers as double values.
Calculate the four arithmetic results.
Use printf() to display the results to two decimal places.
Check whether the divisor is zero before performing division.

Sample run

```
Enter the first number: 20
Enter the second number: 4
Sum: 24.00
Difference: 16.00
Product: 80.00
Quotient: 5.00
```

Test cases

The last test checks that the program handles the zero-divisor case without attempting ordinary floating-point division.

26. Practical 2: Student average calculator

This is an important exercise because it combines variables, data types, keyboard input, arithmetic expressions and formatted output.

The program will:

Ask for a student's name.
Read three scores.
Calculate the total and average.
Display the results.

```
import java.util.Scanner;
public class AverageCalculator {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter student name: ");
String name = input.nextLine();
System.out.print("Enter score for Course 1: ");
double score1 = input.nextDouble();
System.out.print("Enter score for Course 2: ");
double score2 = input.nextDouble();
System.out.print("Enter score for Course 3: ");
double score3 = input.nextDouble();
double total = score1 + score2 + score3;
double average = total / 3.0;
System.out.println("\n--- STUDENT RESULT ---");
System.out.println("Student: " + name);
System.out.printf("Total: %.2f%n", total);
System.out.printf("Average: %.2f%n", average);
}
}
```

Sample run
Enter student name: Amina Bello
Enter score for Course 1: 70
Enter score for Course 2: 80
Enter score for Course 3: 90
--- STUDENT RESULT ---
Student: Amina Bello
Total: 240.00
Average: 80.00

Understand the calculation
The total is:
70 + 80 + 90 = 240
The average is:
240/3 = 80
The program uses double for the scores, total and average so that decimal values can be represented.
Extension exercise: Modify the program to check that each score is between 0 and 100 before calculating the result. Do not assume that user input is valid merely because the variable has a numeric data type.

27. Practical 3: Calculating the total cost of items

This example applies programming to a simple information-system or inventory scenario.

```
import java.util.Scanner;
public class ItemCost {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Enter item name: ");
String itemName = input.nextLine();
System.out.print("Enter quantity: ");
int quantity = input.nextInt();
System.out.print("Enter unit price: ");
double unitPrice = input.nextDouble();
double totalCost = quantity * unitPrice;
System.out.println("\n--- PURCHASE SUMMARY ---");
```

```
System.out.println("Item: " + itemName);
System.out.println("Quantity: " + quantity);
System.out.printf("Unit price: %.2f%n", unitPrice);
System.out.printf("Total cost: %.2f%n", totalCost);
}
}
```

If the quantity is 3 and the unit price is 1500, the total cost is:
3 x 1500 = 4500
The program demonstrates how input data is stored in variables, processed with an arithmetic expression and displayed as output.
For real financial systems, additional validation and appropriate decimal arithmetic would be needed.

Part F — Testing and Debugging

28. What is testing?

Testing involves running a program with selected inputs to determine whether it produces the expected results. For example, test the average calculator with: Testing should cover more than one ordinary example. Include boundary values and invalid inputs where relevant.

29. Common mistakes in Week 3

30. A simple debugging exercise

Consider this program:

```
public class CalculationError {
public static void main(String[] args) {
int firstNumber = 5;
int secondNumber = 2;
double result = firstNumber / secondNumber;
System.out.println(result);
}
}
```

Many beginners expect the output to be 2.5. However, the program displays:
2.0
Why? Both operands are integers, so Java performs integer division first. The integer result is then converted to double.
Correct the expression as follows:

```
double result = (double) firstNumber / secondNumber;
```

Now the output is:
2.5
The cast converts the first operand to double before division.

Part G — Step-by-Step GitHub Submission

31. Prepare your Week 3 project files

Continue using the repository you created in Week 2:
COS201-Programming-I
Do not create a separate repository for every week.
Create a folder named week03 inside your existing local repository.
Your folder structure should be:

```
COS201-Programming-I/
├── README.md
└── week01/
```

```
        └── FirstProgram.java
├── week02/
        ├── HelloWorld.java
        └── StudentProfile.java
└── week03/
        ├── SimpleCalculator.java
        ├── AverageCalculator.java
        └── ItemCost.java
```

Include only the files you have actually created and tested. If you have not completed a particular program, do not create an empty file and present it as finished work.

32. Create and run each Java file

For each program:
Open the cloned COS201-Programming-I folder in VS Code.
Open week03.
Create a new file with the exact required filename.
Copy or type the corresponding Java code.
Save the file.
Open the terminal.
Navigate into week03 if necessary.
Compile the program.
Run the program.
Test it with suitable inputs.
For example, for the average calculator:
javac AverageCalculator.java
java AverageCalculator
For the calculator:
javac SimpleCalculator.java
java SimpleCalculator
For the item-cost program:
javac ItemCost.java
java ItemCost
Run each program from the directory containing its source file, unless you have deliberately configured a different project structure.

33. Update your README.md

Open the main README.md file and add a Week 3 section.
Week 3: Variables, Data Types, Expressions and Input/Output
Objectives
- Declare and initialize Java variables.
- Use appropriate primitive and reference data types.
- Perform arithmetic calculations.
- Read keyboard input using Scanner.
- Display formatted output.
- Compile, test and submit Java programs using GitHub.
Programs

1. `week03/SimpleCalculator.java`

- Performs arithmetic calculations on two numbers.

2. `week03/AverageCalculator.java`

- Calculates the total and average of three scores.

3. `week03/ItemCost.java`
- Calculates the total cost using quantity and unit price.
How to Run
Open a terminal in the `week03` directory.
Compile and run the calculator:
- `javac SimpleCalculator.java`
- `java SimpleCalculator`
Compile and run the average calculator:
- `javac AverageCalculator.java`
- `java AverageCalculator`
Compile and run the item-cost program:
- `javac ItemCost.java`
- `java ItemCost`
Testing
Test each program with different valid inputs and verify
that the results match the expected calculations.
Keep your student details from Week 2 accurate. The README should explain what is actually included in your repository.

34. Commit your Week 3 work
Save all the Java files and README changes in VS Code.
Open GitHub Desktop.
Select your COS201-Programming-I repository.
Open the Changes tab.
Review the files shown.
Confirm that the Week 3 source files and README are included.
Enter the commit summary:
Complete Week 3 variables and input-output practicals

8. Select Commit to main, or the equivalent option for the current branch.
This records your changes locally.

35. Push your work to GitHub
After committing:
Select Push origin in GitHub Desktop.
Wait for the operation to complete.
Open the repository in your browser.
Refresh the page if necessary.
Open week03.
Confirm that all required .java files are visible.
Open each file and check that the code displayed online matches your saved version.
Confirm that the updated README includes the Week 3 section.
If the online files are missing, check whether you committed the correct files and pushed the commit to the remote repository.

36. Submit the repository URL
Copy the actual repository URL from your browser. Do not type an invented link.
Use this submission template:
COS 201 — Week 3 Practical Submission
Dear Lecturer,
I am submitting my Week 3 practical work on Variables, Data Types, Expressions and Input/Output.

Student Name: [Your Full Name]
Matriculation Number: [Your Matric Number]
Programme: [Your Programme]
Course: COS 201 — Computer Programming I
Week: 3
GitHub Repository URL: [Paste your actual repository URL]
Programs submitted:
SimpleCalculator.java
AverageCalculator.java
ItemCost.java
I have saved the source code, tested the programs and pushed the required files to GitHub. The README file contains instructions for compiling and running the programs.
Thank you.

Submit this message through https://docs.google.com/forms/d/e/1FAIpQLScfFS-TTZzdgAZ9qPEFp8gNJpHCu0cYudvHJizNCbKOACAb1A/viewform?usp=header. If your repository is private, ensure that the lecturer has been granted access before the deadline.